

FAST

FORMAÇÃO ACELERADA EM
SOLUÇÕES DE TECHDESIGN



C . E . S . A . R .

Automação de Testes Web

Todos os direitos reservados.

Este material, ou qualquer parte dele, não pode ser reproduzido,
divulgado ou usado de forma alguma sem autorização escrita.



Boas vindas e introdução aos Testes de Automação Web

Automação Web
Aula 1

Olá! Eu sou Thiago Jesus

Engenheiro de Qualidade de Software atuando no Cesar há 4 anos, e com 15 anos de experiência na área. Formado em Análise e Desenvolvimento de Sistemas na Fatec, certificado pela ISTQB com a CTFL (Certified Tester Foundation Level), CTFL-AT (Agile Tester), CTAL-TA (Test Analyst) e CTAL-TM Test Management)



Estrutura do curso

Conteúdo:

Aula 1:

- Fundamentos da Automação de Testes
- Diferenças entre testes manuais e automatizados
- Estrutura de um Script de Teste
- Ferramentas

Aula 2:

- Identificação de Elementos
- Elementos dinâmicos e boas práticas
- Introdução ao padrão page object

Aula 3:

- Organização com page object
- Comandos, esperas e validações
- Comandos, esperas e interações básicas

Aula 4:

- Estrutura de projeto e boas práticas
- Modularização e reuso de código
- Construção de suite de testes

Aula 5:

- Criação de fluxo de teste completo
- Prática guiada
- Bonus: CI/CD + Paralelização

Intervalo: 15 à 20 minutos

Material das aulas: NEO

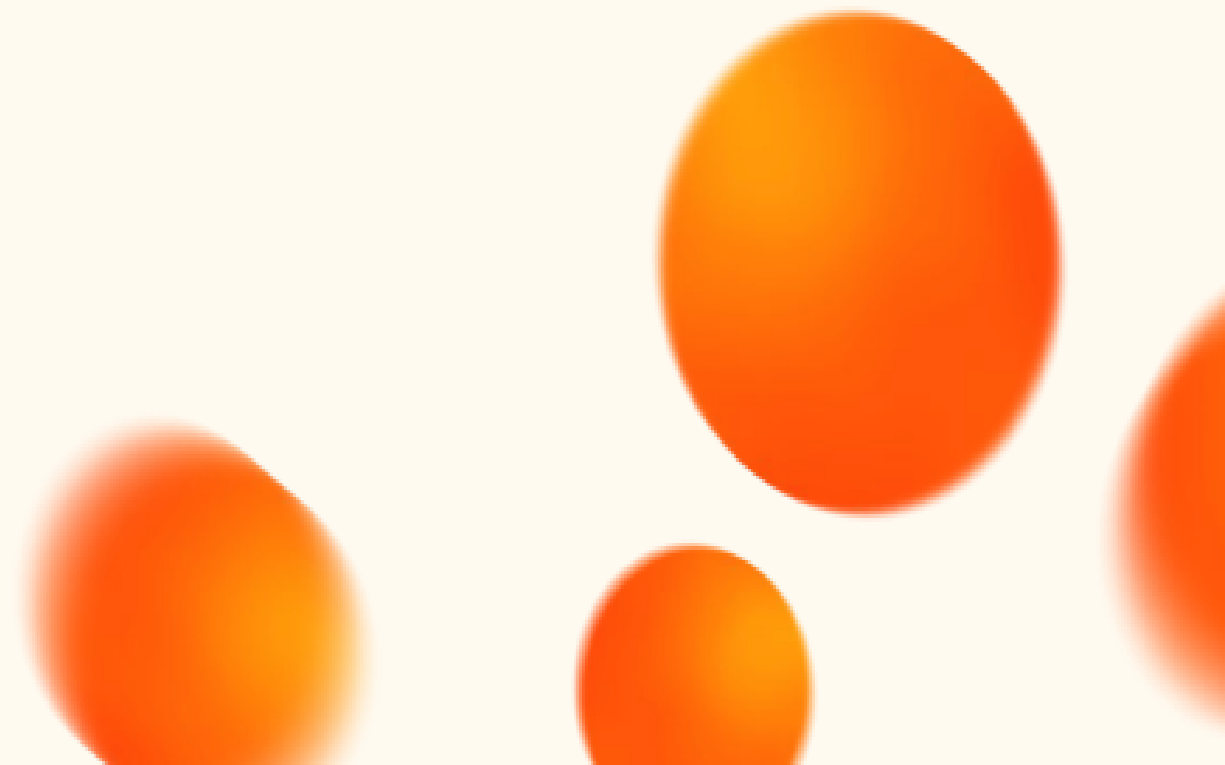
Avaliação Final: 5 questões objetivas

O que veremos hoje?

- Introdução à automação no ciclo de qualidade.
Vantagens, limitações e quando automatizar.
- Comparação prática entre abordagens.
Quando usar cada tipo e como combiná-los.
- Composição básica de um script automatizado.
Etapas: setup, execução, validação e teardown.
- Visão geral e comparação entre ferramentas populares.
- Criação de um script básico com uma das

Você já trabalhou com testes de automação?

Qual linguagem você está usando agora?



O que é um teste automatizado ?

Testes Automatizados

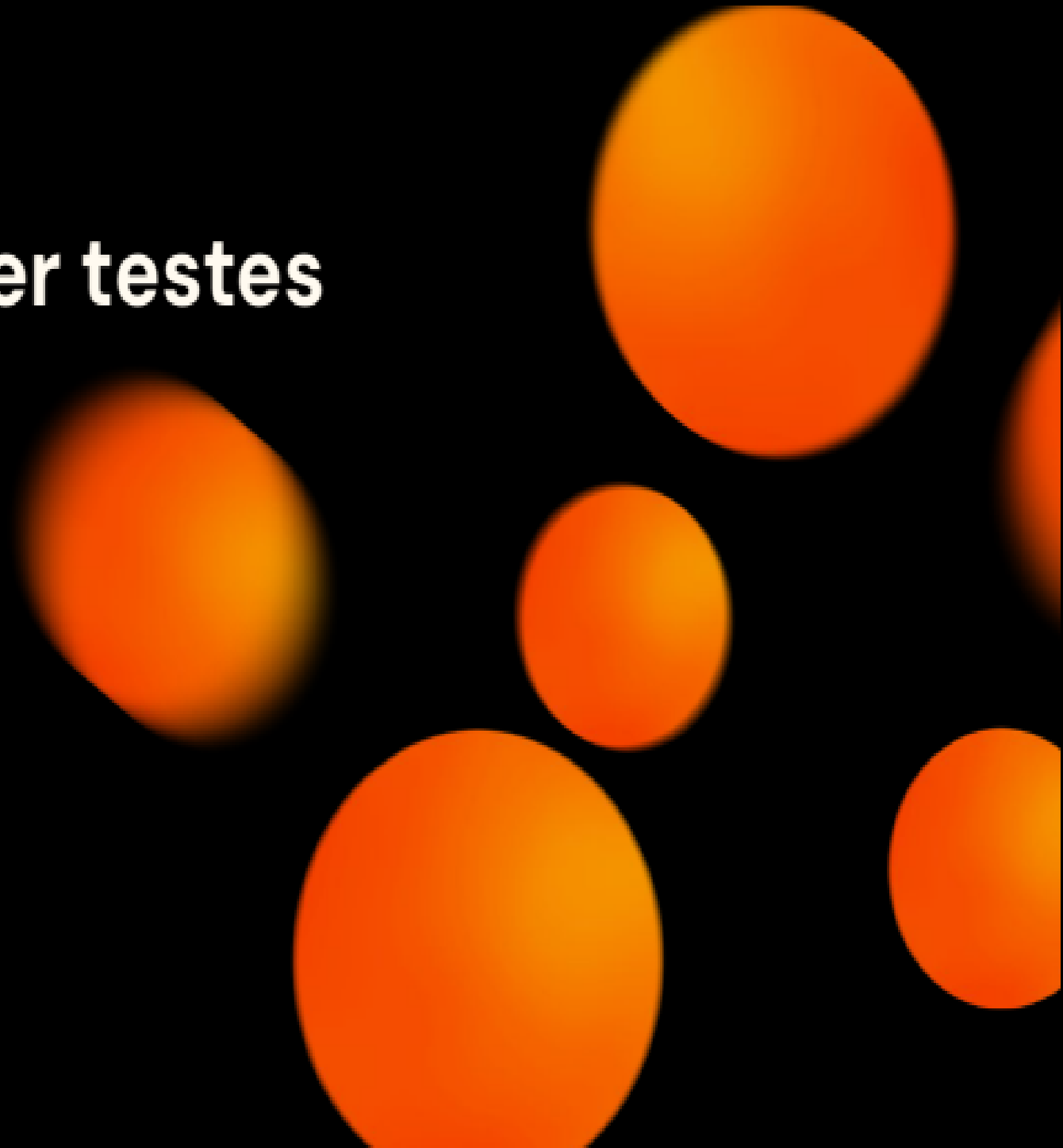
Definição simples:

Testes automatizados são scripts que simulam interações com um sistema para validar se suas funcionalidades estão corretas, sendo executados por ferramentas sem necessidade de ação humana durante o teste.

Tipos comuns de testes automatizados:

Tipo de Teste	O que valida
Unitário (Unit Test)	Funções ou métodos isolados do sistema
Integração	A comunicação entre diferentes módulos ou componentes
Funcional / End-to-End	Fluxos completos do ponto de vista do usuário final
Performance	Tempo de resposta, uso de recursos, estabilidade sob carga

Por que aprender testes automatizados?



The 2025 State of Testing™ Report

11 WHERE DO YOU USE SCRIPTING AND/OR TEST AUTOMATION IN YOUR ORGANIZATION?

**more than one option could be selected*

	2023	2022	2021
For Functional or Regression Testing	77%	73%	75%
For Unit Testing	57%	45%	43%
For Continuous Integration (CI)/Continuous Delivery (CD)	54%	44%	49%
For Load/Stress Testing	38%	31%	43%
For Test data generation	26%	22%	30%
We write BDD/Gherkin scripts using tools like Specflow	21%	17%	20%
For Log, Alerts and Data analysis	20%	15%	14%
We use Home-built test scripts where required	19%	17%	17%
For synthetic and production monitoring and alerts	11%	11%	6%
No, we don't have automation	7%	14%	11%
I don't know where we have automation	1%	2%	2%

The 2025 State of Testing™ Report

12 HOW MUCH HAS AUTOMATION REDUCED YOUR MANUAL TESTING?



The 2025 State of Testing™ Report

18 TESTING SKILLS AND KNOWLEDGE NEEDED TO SUCCEED

*more than one option could be selected

Shifts in importance of skills and knowledge areas needed for testers to thrive in today's testing industry.



Communication skills continue to lead the list of required skills for testing professionals, which is of no surprise, considering its cross-functional importance. Test design emerges as another key factor, which can be explained by the broader involvement of non-dedicated testers, who execute tests that must be designed by testing subject matter experts.

Automação no Ciclo de Qualidade

Vantagens



Ciclo de Qualidade

Automação de Testes no Ciclo de Qualidade de Software

A automação de testes integra-se de forma estratégica ao ciclo de desenvolvimento e qualidade, com o objetivo de validar continuamente o comportamento e a estabilidade do sistema à medida que ele evolui. Ela está presente desde as fases iniciais de desenvolvimento até a entrega e manutenção do produto.

Papel da automação nas etapas do ciclo:

Etapa do Ciclo de Qualidade	Papel da Automação
Planejamento	Definição de testes automatizáveis com base nos requisitos
Desenvolvimento	Escrita de testes unitários e integração junto ao código-fonte
Integração Contínua (CI)	Execução automática de testes a cada novo commit para prevenir quebras no sistema
Testes	Execução de testes funcionais, de performance e regressivos em ambientes controlados
Entrega Contínua (CD)	Validação automática antes de liberar versões para produção
Monitoramento	Testes automatizados em produção para detectar falhas ou lentidão (ex: smoke tests)

Vantagens

Benefícios da automação no ciclo de qualidade:

- **Velocidade nas entregas:** testes rápidos e contínuos.
- **Detecção precoce de falhas:** erros são identificados antes de chegarem à produção.
- **Redução de retrabalho:** menor custo para corrigir defeitos.
- **Cobertura consistente:** cenários importantes são testados em toda versão.
- **Suporte à cultura DevOps:** permite integração e entrega contínuas (CI/CD).

Considerações importantes:

- Nem tudo deve ser automatizado — testes exploratórios e de usabilidade, por exemplo, continuam importantes manualmente.
- A automação exige manutenção — testes frágeis ou mal escritos podem gerar falsos positivos/negativos.
- Deve ser integrada com práticas de qualidade como versionamento, code review e pipelines CI/CD.

Qual a diferença entre testes manuais e automatizados?

Como é a estrutura de cada um?

Manual x Automatizado

Diferença entre Testes Manuais e Automatizados

Aspecto	Testes Manuais	Testes Automatizados
Execução	Realizada por uma pessoa, clicando e interagindo	Executada por scripts ou ferramentas automaticamente
Velocidade	Mais lentos e sujeitos a erros humanos	Muito rápidos e consistentes
Custo inicial	Baixo (não exige codificação)	Alto (exige desenvolvimento e manutenção dos testes)
Manutenção	Pouca ou nenhuma, mas requer mais tempo por execução	Requer manutenção frequente nos scripts
Flexibilidade	Alta — permite testar cenários novos/exploratórios	Limitada ao que foi programado
Indicados para	Testes exploratórios, usabilidade, casos únicos	Testes repetitivos, de regressão, performance, CI/CD
Precisão	Sujeita a variações humanas	Alta precisão e consistência
Retorno sobre tempo	Baixo em execuções únicas	Alto em execuções repetidas ou contínuas



Estrutura do Teste

1. Estrutura de um Teste Manual

Um teste manual é documentado como um caso de teste (test case), geralmente em planilhas, ferramentas de QA (como TestLink, Zephyr, TestRail) ou sistemas de gerenciamento de testes. A estrutura típica inclui:

Elemento	Descrição
ID do Teste	Identificador único (ex: TC-001)
Título	Nome resumido do caso de teste
Pré-condições	Estado necessário do sistema antes da execução
Passos de Teste	Lista sequencial de ações que o testador deve realizar
Dados de Teste	Valores de entrada usados nos passos
Resultado Esperado	O que deve acontecer se o sistema estiver funcionando corretamente
Resultado Real	O que realmente aconteceu (preenchido após a execução)
Status	Aprovado / Reprovado / Bloqueado
Observações	Comentários, prints de erro, bugs encontrados, etc.

Exemplo de Teste Manual

- **ID:** TC-001
- **Título:** Login com credenciais válidas
- **Pré-condição:** Usuário existente no sistema
- **Passos:**
 1. Acessar página de login
 2. Preencher usuário e senha válidos
 3. Clicar em "Entrar"
- **Dados de Teste:** user@example.com / senha123
- **Resultado Esperado:** Usuário é redirecionado para a tela inicial
- **Resultado Real:** (a preencher durante execução)
- **Status:** Aprovado



Estrutura do Teste

2. Estrutura de um Teste Automatizado

Um teste automatizado é escrito em código (geralmente em uma linguagem de programação ou script) e executado por ferramentas como JUnit, Selenium, Cypress, Postman, PyTest, etc. A estrutura segue o padrão AAA (Arrange, Act, Assert):

Bloco (AAA)	Descrição
Arrange (Preparar)	Configurar ambiente, dados e dependências necessárias para o teste
Act (Executar)	Realizar a ação que será testada (ex: enviar uma requisição, clicar botão)
Assert (Verificar)	Comparar o resultado real com o esperado (validação)

Comparando os dois formatos:

Aspecto	Teste Manual	Teste Automatizado
Formato	Documento com passos	Código (script ou função)
Execução	Manual, feita por uma pessoa	Automática, feita por ferramenta
Registro de resultado	Manual, preenchido após execução	Gerado automaticamente (log, relatório, print)
Reusabilidade	Limitada	Alta (pode rodar milhares de vezes)

Exemplo de Teste Automatizado (Java + JUnit)

```
java

@Test
public void testLoginComCredenciaisValidas() {
    // Arrange
    LoginPage login = new LoginPage(driver);
    login.abrirPagina();

    // Act
    login.preencherUsuario("user@example.com");
    login.preencherSenha("senha123");
    login.clicarEntrar();

    // Assert
    assertTrue(login.foiRedirecionadoParaPaginaInicial());
}
```

Gherkin

Cenário Web (Login) — Escrita em Gherkin

gherkin

Feature: Autenticação de usuário

Como um usuário registrado
Quero fazer login no sistema
Para acessar a área restrita

Scenario: Login com credenciais válidas

Given que o usuário está na página de login
When ele preenche o campo "usuário" com "user@example.com"
And preenche o campo "senha" com "senha123"
And clica no botão "Entrar"
Then ele deve ser redirecionado para a página inicial
And deve ver a mensagem "Ben-vindo, usuário!"

O que esse Gherkin representa?

Parte	Função
Feature	Descreve o que está sendo testado (login, no caso)
Scenario	Caso de uso específico com fluxo completo
Given	Pré-condição — onde o usuário está
When	Ações do usuário
Then	Resultados esperados

Gherkin + Selenium

Exemplo de implementação (usando Cucumber + Selenium + Java)

```
java Copy Edit

@Given("que o usuário está na página de login")
public void usuario_esta_na_pagina_de_login() {
    driver.get("https://meusistema.com/login");
}

@When("ele preenche o campo {string} com {string}")
public void preencher_campo_com(String campo, String valor) {
    if (campo.equals("usuario")) {
        driver.findElement(By.id("username")).sendKeys(valor);
    } else if (campo.equals("senha")) {
        driver.findElement(By.id("password")).sendKeys(valor);
    }
}

@When("clica no botão {string}")
public void clica_no_botao(String botoao) {
    if (botoao.equals("Entrar")) {
        driver.findElement(By.id("login-button")).click();
    }
}

@Then("ele deve ser redirecionado para a página inicial")
public void redirecionado_para_home() {
    new WebDriverWait(driver, Duration.ofSeconds(10))
        .until(ExpectedConditions.urlContains("/home"));
}

@Then("deve ver a mensagem {string}")
public void deve_ver_mensagem(String mensagemEsperada) {
    String mensagem = driver.findElement(By.id("welcome-message")).getText();
    assertEquals(mensagemEsperada, mensagem);
}
```

Ferramentas envolvidas:

- Gherkin: linguagem dos cenários (.feature)
- Cucumber: framework que interpreta os arquivos .feature
- Selenium WebDriver: automação do navegador
- JUnit/TestNG: execução dos testes
- Maven/Gradle: build e dependências

INTERVALO!

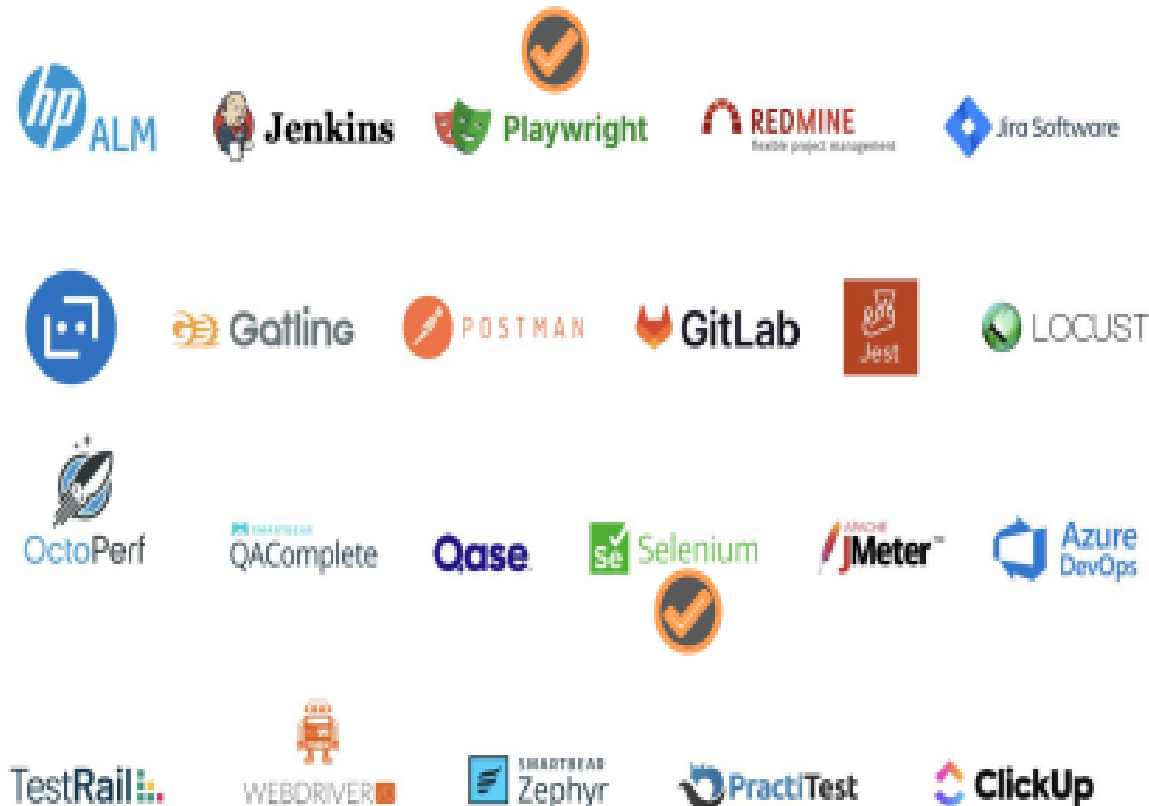


20 min.

The 2025 State of Testing™ Report

15 YOUR TECH STACK:

Which testing-related tools does your organization use to support the QA process?



Selenium x Playwright x Cypress

✓ 1. Linguagens Suportadas

- **Selenium:** Java, Python, C#, JavaScript, Ruby, Kotlin
- **Playwright:** JavaScript/TypeScript, Python, Java, .NET
- **Cypress:** JavaScript/TypeScript (focado no ecossistema Node.js)

Selenium x Playwright x Cypress

✓ 2. Arquitetura e Execução

- Selenium:
 - Baseado em **WebDriver**, interage com o navegador via drivers oficiais.
 - Necessita de configuração extra (drivers, Selenium Grid para paralelismo).
- Playwright:
 - **Headless-first**, mas suporta modo visível.
 - Instala navegadores **bundled** (Chromium, Firefox, WebKit).
 - Paralelismo nativo e isolamento de contexto.
- Cypress:
 - Roda **dentro do navegador**, compartilhando o mesmo ciclo de eventos da aplicação.
 - Não é baseado em WebDriver.
 - Fácil configuração, mas limitada a navegadores baseados em Chromium e Firefox (suporte experimental ao WebKit).

Selenium x Playwright x Cypress

✓ 3. Velocidade de Execução

- **Selenium:** Mais lento, pois depende do protocolo WebDriver.
- **Playwright:** Muito rápido (comunicação direta com o browser via protocolo DevTools).
- **Cypress:** Rápido, mas um pouco mais lento que Playwright para cenários muito grandes.

Selenium x Playwright x Cypress

✓ Comparativo: Selenium vs Playwright vs Cypress

Critério	Selenium	Playwright	Cypress
Linguagens suportadas	✓ Ample suporte: Java, Python, C#, JS, Ruby, Kotlin	JS/TS, Python, Java, .NET	JS/TS (Node.js)
Maturidade e Comunidade	✓ Mais maduro e consolidado (desde 2004), padrão da indústria	Projeto mais recente (2020), mantido pela Microsoft	Comunidade forte, mas restrita ao ecossistema JS
Execução Cross-Browser	✓ Suporte nativo e completo a todos os navegadores	Suporte completo, com navegadores internos	⚠ Chromium, Firefox, WebKit (em beta)
Suporte a múltiplas plataformas	✓ Windows, Linux, macOS, até emuladores mobile e navegadores remotos	Compatível também, mas requer mais configuração	Limitado a ambiente desktop
Integração com outras ferramentas	✓ Alta compatibilidade com frameworks de testes, CI/CD e nuvens	Bom compatibilidade	Bom integração, mas menor flexibilidade externa
Ambientes corporativos e legados	✓ Ideal para sistemas antigos e complexos	Mais voltado a apps modernas	Voltado para SPAs modernas (React, Vue, Angular)
Execução distribuída (Grid)	✓ Selenium Grid para testes em paralelo em múltiplos ambientes	Suporte nativo, com múltiplas instâncias	Paralelismo via Dashboard (licença paga)
Curva de aprendizado	⚠ Mais técnica, mas com vasto material de apoio	Médio	Muito fácil para quem já usa JS
Auto-wait e espera inteligente	✗ Manual ou via bibliotecas extras	✓ Nativo	✓ Nativo
Facilidade de setup	⚠ Requer configuração de drivers	Instala navegadores automaticamente	Setup muito simples

Selenium x Playwright x Cypress

✓ 5. Comunidade e Maturidade

- **Selenium:** Muito madura, enorme comunidade, padrão da indústria.
 - **Playwright:** Mais recente (by Microsoft), crescendo rápido, ótima documentação.
 - **Cypress:** Popular para front-end moderno, ótima DX (Developer Experience), mas menos flexível.
-

✓ 6. Casos de Uso Indicados

- **Selenium:** Ambientes legados, múltiplas linguagens, alta compatibilidade.
- **Playwright:** Projetos modernos, alta velocidade, testes cross-browser robustos.
- **Cypress:** Aplicações front-end JS/TS, equipes que querem fácil integração com CI/CD.

Selenium



Prós em Destaque do Selenium

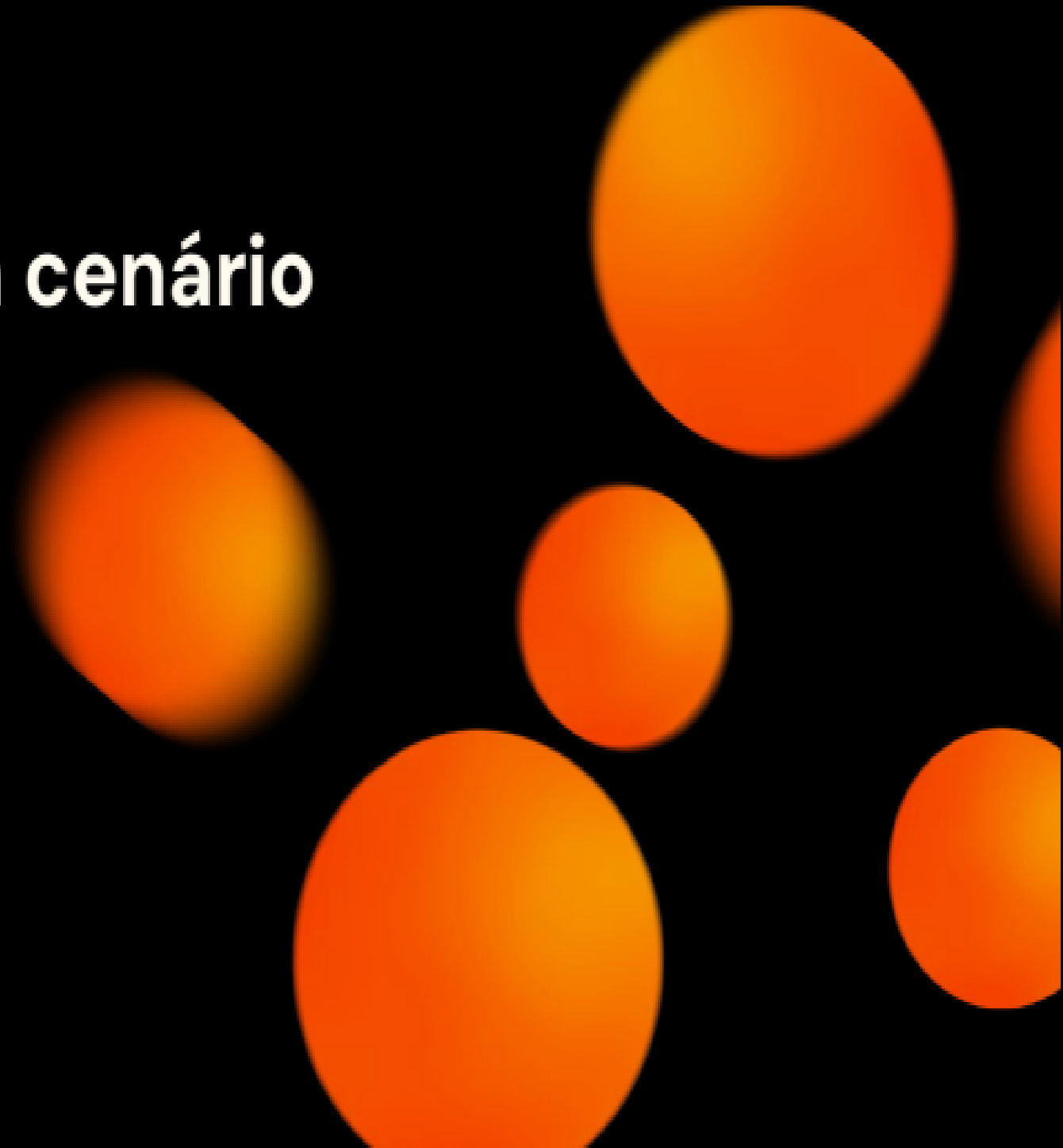
- **◆ Padrão de mercado** para automação web há anos.
- **◆ Flexibilidade de linguagem** (pode ser usado em praticamente qualquer stack).
- **◆ Ampla adoção em empresas de grande porte**, especialmente com aplicações legadas.
- **◆ Integração com serviços de nuvem** como BrowserStack, Sauce Labs, LambdaTest etc.
- **◆ Alta personalização e controle de execução** (browser profiles, proxies, etc).
- **◆ Compatível com testes mobile via Appium**, ampliando seu uso além da web.

Nosso Framework

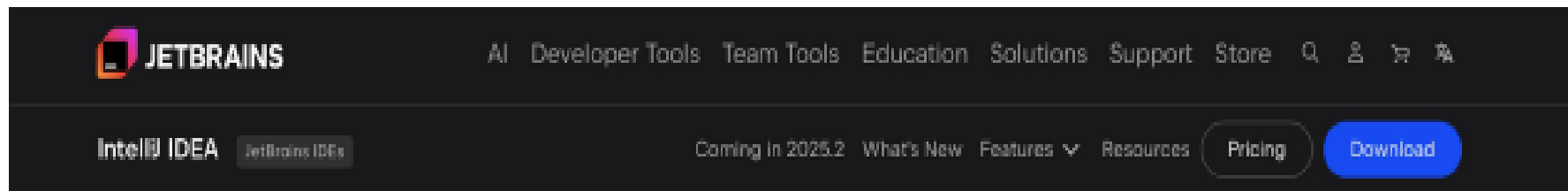
Papéis de Cada Tecnologia no Framework

Componente	Papel no Framework
Java	Linguagem principal utilizada para escrever o código dos testes, steps, page objects e lógica de automação.
Maven	Gerenciador de dependências e build. Automatiza a importação de bibliotecas (como Selenium, Cucumber e JUnit), compila o projeto e executa os testes.
Cucumber	Permite escrever os cenários de teste em linguagem natural (Gherkin), promovendo BDD (Behavior-Driven Development). Facilita a colaboração entre QA, devs e stakeholders.
Selenium WebDriver	Realiza a automação no navegador. Simula ações do usuário (clique, digitação, navegação etc.) interagindo diretamente com a interface web.
JUnit	Framework de testes utilizado como executor dos testes . Integra com o Cucumber para rodar os cenários automaticamente e gerar relatórios.

Começando um cenário automatizado



Download IDE



Windows macOS Linux

IntelliJ IDEA Ultimate

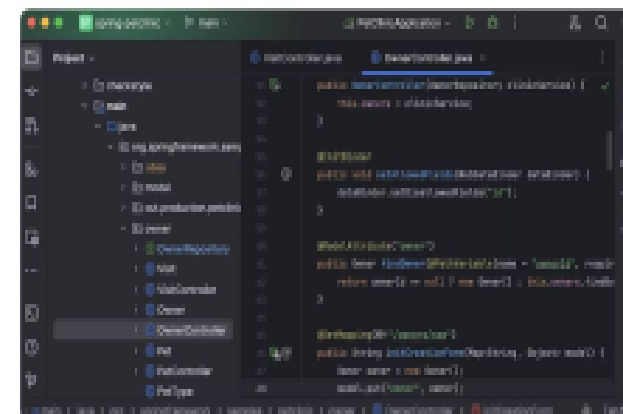
The IDE for Professional Development in
Java and Kotlin

Download

.dmg (Apple Silicon) ▼

Free 30-day trial

 Select an installer for Intel or Apple Silicon



Version: 2025.1.4.1

Built: 251.27812.49

24 July 2025

[System requirements](#)

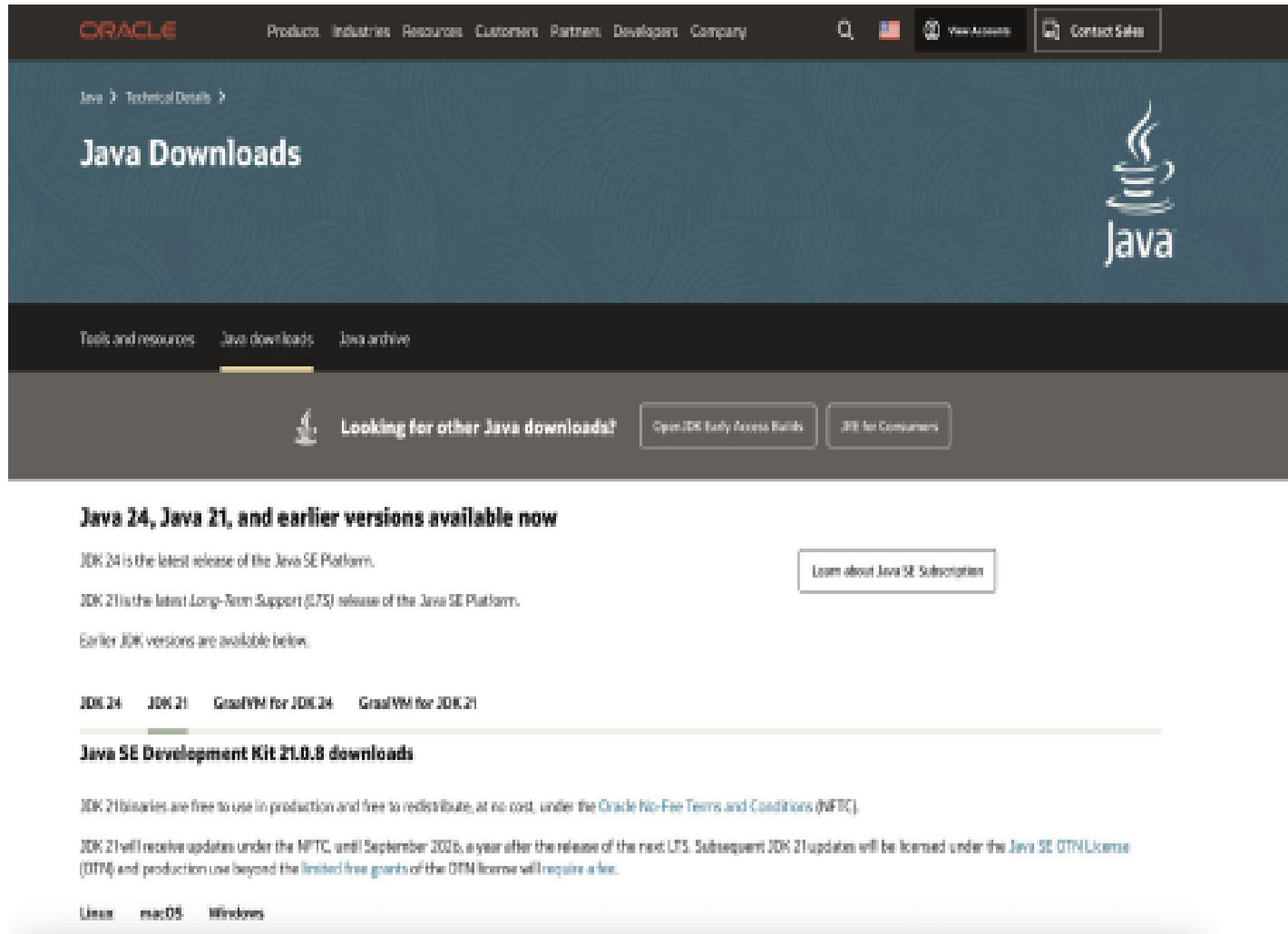
[Installation instructions](#)

[Other versions](#)

[Third-party software](#)

<https://www.jetbrains.com/idea/download/>

Download JDK



The screenshot shows the Oracle Java Downloads page. The header includes the Oracle logo and navigation links: Products, Industries, Resources, Customers, Partners, Developers, and Company. There are also links for 'View Account' and 'Contact Sales'. The main heading is 'Java Downloads' with a Java logo. Below this, there are tabs for 'Tools and resources', 'Java downloads' (which is active), and 'Java archive'. A section titled 'Looking for other Java downloads?' includes links for 'Open JDK Early Access Builds' and 'JRE for Consumers'. The main content area features the heading 'Java 24, Java 21, and earlier versions available now'. It states that JDK 24 is the latest release of the Java SE Platform and JDK 21 is the latest Long-Term Support (LTS) release. A link 'Learn about Java SE Subscriptions' is provided. Below this, there are links for 'JDK 24', 'JDK 21', 'GraalVM for JDK 24', and 'GraalVM for JDK 21'. The 'JDK 21' link is highlighted. The section 'Java SE Development Kit 21.0.8 downloads' follows, stating that JDK 21 binaries are free to use in production and free to redistribute under the Oracle No-Fee Terms and Conditions (NFTC). It also mentions that JDK 21 will receive updates under the NFTC until September 2026, a year after the release of the next LTS. Subsequent JDK 21 updates will be licensed under the Java SE OTN License (OTN) and production use beyond the limited free grants of the OTN license will require a fee. At the bottom, there are links for 'Linux', 'macOS', and 'Windows'.

Java 24, Java 21, and earlier versions available now

JDK 24 is the latest release of the Java SE Platform.

JDK 21 is the latest Long-Term Support (LTS) release of the Java SE Platform.

Earlier JDK versions are available below.

[JDK 24](#) [JDK 21](#) [GraalVM for JDK 24](#) [GraalVM for JDK 21](#)

Java SE Development Kit 21.0.8 downloads

JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions (NFTC).

JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subsequent JDK 21 updates will be licensed under the Java SE OTN License (OTN) and production use beyond the limited free grants of the OTN license will require a fee.

[Linux](#) [macOS](#) [Windows](#)

<https://www.oracle.com/java/technologies/downloads/?er=221886#java21>

Arquivo Pom

```
m pom.xml (FAST2025) x
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
6
7      <groupId>org.example</groupId>
8      <artifactId>FAST2025</artifactId>
9      <version>1.0-SNAPSHOT</version>
10     <name>FAST2025</name>
11
12     <build>
13         <plugins>
14             <plugin>
15                 <groupId>org.apache.maven.plugins</groupId>
16                 <artifactId>maven-compiler-plugin</artifactId>
17                 <version>3.10.1</version>
18                 <configuration>
19                     <encoding>UTF-8</encoding>
20                     <source>18</source>
21                     <target>18</target>
22                 </configuration>
23             </plugin>
24         </plugins>
25     </build>
26
```

Arquivo Pom

```
27     <dependencies>
28         <dependency>
29             <groupId>org.seleniumhq.selenium</groupId>
30             <artifactId>selenium-java</artifactId>
31             <version>4.19.0</version>
32         </dependency>
33         <dependency>
34             <groupId>org.seleniumhq.selenium</groupId>
35             <artifactId>selenium-chrome-driver</artifactId>
36             <version>4.19.0</version>
37         </dependency>
38         <dependency>
39             <groupId>io.cucumber</groupId>
40             <artifactId>cucumber-java</artifactId>
41             <version>6.1.2</version>
42         </dependency>
43         <dependency>
44             <groupId>io.cucumber</groupId>
45             <artifactId>cucumber-junit</artifactId>
46             <version>7.21.1</version>
47         </dependency>
48     </dependencies>
49 </project>
```

DÚVIDAS?!
:)





C . E . S . A . R

MUITO OBRIGADO!

© CESAR | Todos os direitos reservados